UNIT-4
Schema Refinement

Problems Caused by Redundancy

Storing the same information **redundantly**, that is, in more than one place within a database, can lead to several problems:

1. **Update anomalies**
2. **Insertion anomalies**
3. **Deletion anomalies**

Insert Anomaly

An **Insert Anomaly** occurs when certain attributes cannot be inserted into the database without the presence of other attributes.

Course _no	Tutor	Room	Room_size	En_limit
353	Smith	A532	45	40
351	Smith	C320	100	60
355	Clark	H940	400	300
456	Turner	H940	400	45

e.g. we have built a new room (e.g. B123) but it has not yet been timetabled for any courses or members of staff.

Delete Anomaly

A **Delete Anomaly** exists when certain attributes are lost because of the deletion of other attributes.

Course_no	Tutor	Room	Room_size	En_limit
353	Smith	A532	45	40
351	Smith	C320	100	60
355	Clark	H940	400	300
456	Turner	H940	400	45

e.g. if we remove the entity, course_no:351 from the above table, the details of room C320 get deleted. Which implies the corresponding course will also get deleted.

Update Anomaly

An **Update Anomaly** exists when one or more instances of duplicated data is updated, but not all.

Course_no	Tutor	Room	Room_size	En_limit
353	Smith	A532	45	40
351	Smith	C320	100	60
355	Clark	H940	400	300
456	Turner	H940	400	45

e.g. Room H940 has been improved, it is now of RSize = 500. For updating a single entity, we have to update all other columns where room=H940.

Insert Anomaly

An **Insert Anomaly** occurs when certain attributes cannot be inserted into the database without the presence of other attributes.

Student

ID	Name	Age	Branch	Branch_Code	Hod_name
1	A	17	Civil	101	Aman
2	B	18	Civil	101	Aman
3	C	19	Civil	101	Aman
4	D	20	CS	102	Monu
5	E	21	CS	102	Monu
6	F	22	Electrical	103	Rakesh

For example, let's take a branch name petroleum, now the data regarding petroleum cannot be stored in the table unless we insert a student which is in petroleum.

Delete Anomaly

A **Delete Anomaly** exists when certain attributes are lost because of the deletion of other attributes.

For example, suppose a **student of the electrical branch is leaving so now** we have to delete the data of that student, but the problem is if we delete the student data, then branch data will also get deleted along with that as there is only one student present through which branch data is present.

ID	Name	Age	Branch	Branch_Code	Hod_name
1	A	17	Civil	101	Aman
2	B	18	Civil	101	Aman
3	C	19	Civil	101	Aman
4	D	20	CS	102	Monu
5	E	21	CS	102	Monu

Functional dependency in DBMS (FD's)

Functional dependency(FD)

A functional dependency is an **association between two attributes** of the same relational database table.

Given a relation *R*, a set of attributes *X* in *R* is said to **functionally determine** another attribute *Y*, also in *R*, (written $X \rightarrow Y$) if and only if each *X* value is associated with **at most one *Y* value.**

1. X is a determinant
2. X determines Y
3. Y is functionally dependent on X
4. $X \rightarrow Y$

For example:

For example:

Suppose we have a student table with attributes:

Stu_Id, Stu_Name, Stu_Age.

Here **Stu_Id** attribute uniquely identifies the **Stu_Name** attribute of student table because if we know the student id we can tell the student name associated with it.

For example:

Functional dependency and can be written as :

Stu_Id >Stu_Name _

we can say **Stu_Name** is functionally dependent on **Stu_Id.**

Formally:

If column A of a table uniquely identifies the column B of same table then it can represented as A>B (Attribute B is functionally dependent on attribute A)

Example

Employee

SSN	Name	JobType	DeptName
557-78-6587	Lance Smith	Accountant	Salary
214-45-2398	Lance Smith	Engineer	Product

Note: Name is functionally dependent on SSN because an employee's name can be uniquely determined from their SSN.

Name does not determine SSN, because more than one employee can have the same name..

Example

.

The following table illustrates A ⟹ B:

A	B
1	1
2	4
3	9
4	16
2	4
7	9

Since for each value of A there is associated one and only one value of B

Example

The following table illustrates that A does not functionally determine B:

A	B
1	1
2	4
3	9
4	16
3	10

Since for A = 3 there is associated more than one value of B.

Example: Consider the database having following tables:

The Supplier Table

SNo.	Sname	Status	City
S1	Suneet	20	Qadlan
S2	Ankit	10	Amritsar
S3	Amit	10	Amritsar

Here, Sname is FD on Sno. Because, Sname can take only one value for the given value of Sno (**Sno -> Sname**)

Similarly, city and status are also FD on Sno, because for each value of Sno there will be only one city and status.

FD is represented as:

Sno > City

Sno >Status

S. Sno > S (Sname, City, Status).etc

Consider another database of shipment with following attributes:

The Shipment Table

SNo.	Pno.	Qty
S1	P1	270
S1	P2	300
S1	P3	700
S2	P1	270
S2	P2	700
S3	P2	300

In this case Qty is FD on combination of Sno, Pno because each combination of Sno and Pno results only for one Quantity.

SP (Sno, Pno) —> SP.QTY

Identify FD'S

The Part Table

PNo.	Pname	Color	Weight	City
P1	Nut	Red	12	Qadlan
P2	Bolt	Green	17	Amritsar
P3	Screw	Blue	17	Amritsar
P4	Screw	Red	14	Qadlan

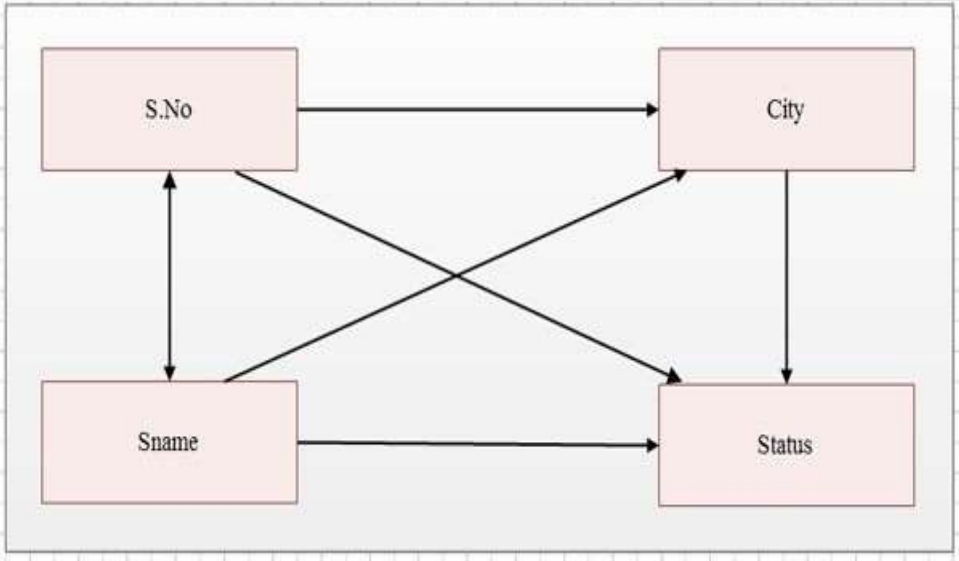
Dependency Diagrams

1. A dependency diagram consists of **the attribute names and all functional dependencies** in a given table.

The dependency diagram of Supplier table is.

Here, following functional dependencies exist in supplier table

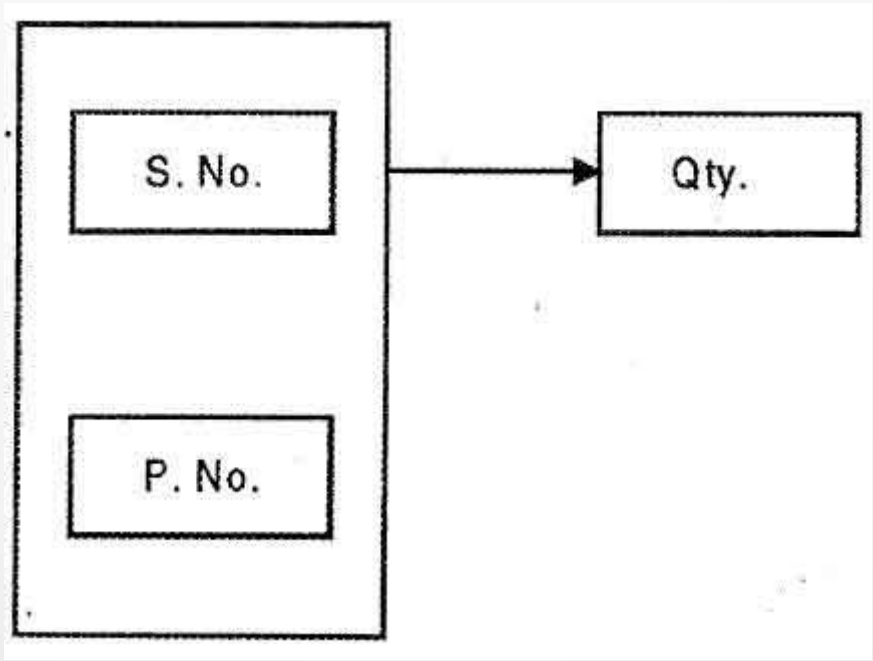
Sno	**Sname**
Sname	**Sno**
Sno	**City**
Sno	**Status**
Sname	**City**
Sname	**Status**
City	**Status**



The FD diagram of relation Shipment is

Here following functional dependencies exist in parts table

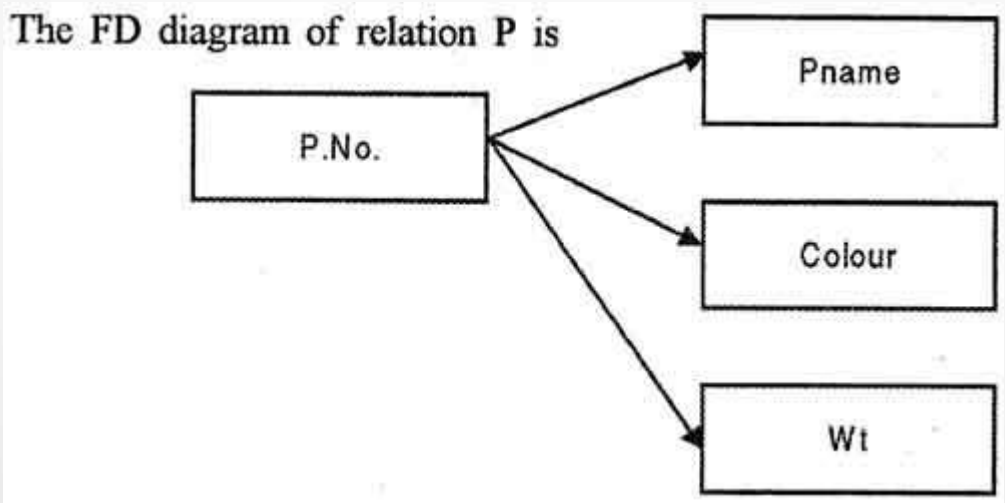
SP (Sno, Pno) –SP.QTY



The FD diagram of relation P is

Here following functional dependencies exist in Part table:

Pno –Pname
Pno –Color
Pno –Wt



EXAMPLE:

A Relation R(A,B,C) Having The Following Tuples
R(A,B,C)
(1,2,3)
(4,2,3)
(5,3,3)
Then Which Of The Following FD Is Not Valid?

a)A>B

b)BC>A

a)B>C

b)AC>B

b)BC>A

Advantages of Functional Dependency

1. Functional Dependency **avoids data redundancy where same data should not be repeated** at multiple locations in same database.

2. It **maintains the quality** of data in database.

3. It allows **clearly defined meanings and constraints** of databases.

4. It helps in **identifying bad designs**.

5. It expresses the facts about the database design.

Armstrong's Axioms

(Inference rules for FDs)

Introduction to Axioms Rules

1. Armstrong's Axioms is a **set of rules.**
2. It provides a simple technique for reasoning about functional dependencies.
3. It was developed by William W. Armstrong in 1974.
4. It is used to infer all the functional dependencies on a relational database.

Armstrong's Inference Rules –

Let A, B and C and D be arbitrary subsets of the set of attributes of the giver relation R, and let AB be the union of A and B. Then,⇒→

A. Primary Rules

1. **Reflexivity :**
   If B is subset of A, then A → B

2. **Augmentation :**
   If A → B, then AC → BC

3. **Transitivity :**
   If A → B and B → C, then A → C.

Armstrong's Inference Rules –

B. Secondary Rules

1. **Projectivity or Decomposition Rule :**
   If A → BC, Then A → B and A → C

2. **Union or Additive Rule :**
   If A→B, and A→C Then A→BC.

3. **Pseudo Transitive Rule :**
   If A → B, DB → C, then DA → C

Types of Functional Dependencies

1. **Trivial functional dependency**
2. **Non-trivial functional dependency**

Trivial	If A holds B {A → B}, where B is a subset of A, then it is called a **Trivial Functional Dependency**. Trivial always holds Functional Dependency.
Non-Trivial	If A holds B {A → B}, where B is not a subset A, then it is called as a **Non-Trivial Functional Dependency.**
Completely Non-Trivial	If A holds B {A → B}, where A intersect B = Φ, it is called as a **Completely Non-Trivial Functional Dependency.**

Trivial functional dependency

The **dependency of an attribute on a set of attributes** is known as trivial functional dependency if the set of attributes includes that attribute.

Symbolically:

A >B is trivial functional dependency if B is a subset of A.

The following dependencies are also trivial: A>A & B>B

For example:

For example:

Consider a table with two columns **Student_id** and **Student_Name**.

{**Student_Id, Student_Name**} > **Student_Id** is a trivial functional dependency as **Student_Id** is a subset of {**Student_Id, Student_Name**}.

Also,
Student_Id > Student_Id & Student_Name > Student_Name are trivial dependencies too.

Non-trivial functional dependency

If a functional dependency X–>Y holds true where Y is not a subset of X then this dependency is called non trivial Functional dependency.

Example :

An employee table with three attributes: **emp_id, emp_name, emp_address**.

The following functional dependencies are nontrivial:

emp_id > emp_name (**emp_name** is not a subset of **emp_id**)
emp_id > emp_address (**emp_address** is not a subset of **emp_id**)

Completely non trivial FD:

On the other hand, the following dependencies are trivial:

{emp_id, emp_name} > emp_name

[emp_name is a subset of {emp_id, emp_name}]

Completely non trivial FD:

If a Functional dependency **X->Y** holds true where **X** intersection **Y** is **Null** then this dependency is said to be completely **non trivial function dependency**.

Question 1:

Consider a relational scheme R with attributes A,B,C,D,F and the FDs

A → BC

B → E

CD → EF

Prove that functional dependency AD → F holds in R.

Step 1 : A → BC (Given)

Step 2 : A → C (Decomposition Rule applied on step 1)

Step 3 : AD → CD (Augmentation Rule applied on step 2)

Step 4 : CD → EF (Given)

Step 5 : AD → EF (transivity Rule applied on step 3 and 4)

Step 6 : AD → F (Decomposition Rule applied on step 5)

Example:

Consider relation E = (P, Q, R, S, T, U) having set of Functional Dependencies (FD).

$P \rightarrow Q$ $P \rightarrow R$

$QR \rightarrow S$ $Q \rightarrow T$

$QR \rightarrow U$ $PR \rightarrow U$

Calculate some members of Axioms are as follows,

1. $P \rightarrow T$
2. $PR \rightarrow S$
3. $QR \rightarrow SU$
4. $PR \rightarrow SU$

SolutionS:

1. **P → T**

   In the above FD set, P → Q and Q → T

   So, Using Transitive Rule: If {A → B} and {B → C}, then {A → C}

   ∴ If P → Q and Q → T, then **P → T.**

2. **PR → S**

In the above FD set, P → Q
As, QR → S

So, Using Pseudo Transitivity Rule: If{A → B} and {BC → D}, then {AC → D}

∴ If P → Q and QR → S, **then PR → S.**

SolutionS:

3. QR → SU

In above FD set, QR → S and QR → U

So, Using Union Rule: If{A → B} and {A → C}, then {A → BC}

∴ If QR → S and QR → U, **then QR → SU.**

4. PR → SU

So, Using Pseudo Transitivity Rule: If{A → B} and {BC → D}, then {AC → D}

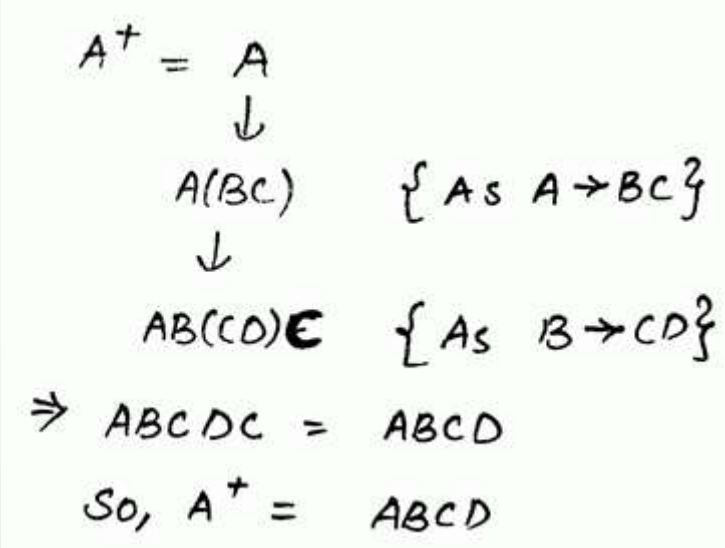
∴ If PR → S and PR → U, **then PR → SU.**

Closure of Attribute.

Closure of Attribute:

1. All attributes derived by using a given attribute called as closure of attributes.
2. It is represented in A+.

Example:



Example:

Suppose we are given relation R with attributes A,B,C,D and FDs {A→BC B→CD}

To find the Closure of all the attributes :

Let us find closure of A firstly.

$A^+ = A$

$\downarrow$

$A(BC)$ {As $A \rightarrow BC$}

$\downarrow$

$AB(CD) \in$ {As $B \rightarrow CD$}

$\Rightarrow ABCDC = ABCD$

So, $A^+ = ABCD$

Example:

To find the Closure of all the Remaining attributes :

$(B^+) = \{BCD\}$

$(C^+) = \{C\}$

$(D^+) = \{D\}$

$(AB^+) = \{ABCD\}$

$(AC^+) = \{ABCD\}$

$(AD^+) = \{ABCD\}$

$(BC^+) = \{BCD\}$

$(BD^+) = \{BCD\}$

$(CD^+) = \{CD\}$

$(ABC^+) = \{ABCD\}$

$(ABD^+) = \{ABCD\}$

$(ACD^+) = \{ABCD\}$

$(BCD^+) = \{BCD\}$

$(ABCD^+) = \{ABCDEF\}$

Example.

Example. Assume that there are 4 attributes A, B, C, D, and that
$F = \{ A \rightarrow B, B \rightarrow C\}$.
To compute $F+$, we first get:

1. $A+ = AB+ = AC+ = ABC+ = \{A, B, C\}$
2. $B+ = BC+ = \{B, C\}$
3. $C+ = \{C\}$
4. $D+ = \{D\}$
5. $AD+ = \{A, D\}$
6. $BC+ = \{B, C\}$
7. $BD+ = BCD+ = \{B, C, D\}$
8. $ABD+ = ABCD+ = \{A, B, C, D\}$
9. $ACD+ = \{A, C, D\}$

It is easy to generate the FDs in $F+$ from the closures of the above attribute sets.

Question :

Compute the closure of following set F of functional dependencies for relation schema

R = {A, B, C, D, E}.

A → BC ,CD → E, B → D, E → A

To find the Closure of all the attributes :

Solution :

$(A)^+ = \{ABCDE\}$

$(C)^+ = \{C\}$

$(D)^+ = \{D\}$

$(AB)^+ = \{ABCDE\}$

$(AC)^+ = \{ABCDE\}$

$(AD)^+ = \{ABCDE\}$

$(AE)^+ = \{ABCDE\}$

$(BC)^+ = \{ABCDE\}$

$(BD)^+ = \{BD\}$

$(BE)^+ = \{ABCDE\}$

$(CD)^+ = \{ABCDE\}$

$(CE)^+ = \{ABCDE\}$

$(DE)^+ = \{ABCDE\}$……

Question :

Following dependencies are given:

AB → CD, AF → D, DE → F, C → G, F → E, G → A

Which of the following is false ?

(a) $(CF)^+ = \{ACDEFG\}$

(b) (b) $(BG)^+ = \{ABCDG\}$

(c) (c) $(AF)^+ = \{ACDEFG\}$

(d) (d) $(AB)^+ = \{ABCDG\}$

Solution :

option (c) is false . as $(AF)^+$ will give {AFED} but not {ACDEFG}

Closure of Functional Dependency

Closure of Functional Dependency

A Closure is a set of FDs is a set of all possible FDs that can be derived from a given set of FDs.

let F be the set of FDs we have collected. Then:

Definition **The closure of F, denoted as F+,** is the set of all regular FDs that can be derived from F.

Do not confuse the closure of F with the closure of an attribute set.

Example.

Assume that there are 4 attributes A, B, C, D, and that
F = { A → B, B → C} .

Then, F + includes all the following FDs:

{A → A, A → B, A → C, B → B, B → C, C → C, D → D, AB → A, AB → B,
AB → C, AC → A, AC → B, AC → C, AD → A, AD → B, AD → C,
AD → D, BC → B, BC → C, BD → B, BD → C, BD → D, CD → C,
CD → D, ABC → A, ABC → B, ABC → C, ABD → A, ABD → B,
ABD → C, ABD → D, BCD → B, BCD → C, BCD → D, ABCD → A,
ABCD → B, ABCD → C, ABCD → D.}

Applications of closure of attribute

Applications of closure of attribute

There are 2 applications for closure of attribute

1.Finding additional FD's

2.Finding key for a relation.

Applications of closure of attribute

R(A,B,C) where F={A>B,B>C}
Find A>C is valid over R

A^+ ={A,B,C}

A>C is valid over R

Database Decomposition

What is decomposition?

Decomposition of a Relation

Definition: The process of **breaking up or dividing a single relation** into two or more sub relations is called as decomposition of a relation.

1. It should **always be lossless**, because it confirms that the information in the original relation can be accurately reconstructed based on the decomposed relations.

2. If there is **no proper decomposition** of the relation, then it may lead to problems like **loss of information**.

Properties of Decomposition

Following are the properties of Decomposition,

1. **Lossless Join Decomposition Property**

2. **Dependency Preserving Property**

Lossless Join Decomposition

Definition :

Let ***R*** be the relational schema with instance ***r*** is decomposed into ***R1,R2,….,Rn*** with instance ***r1,r2,…..,rn.***

If $R1 \bowtie R2 \bowtie \ldots\ldots \bowtie Rn = R$, then it is called Lossless Join Decomposition.

i.e. if **natural joins** of all the decompositions gives the original relation, then it is said to be Lossless Join Decomposition.

Lossless Join Decomposition

We can follow certain rules to ensure that the decomposition is a lossless join decomposition Let's say we have a relation R and we decomposed it into R1 and R2, then the rules are:

1. **The union of attributes of both the sub relations R1 and R2 must contain all the attributes of original relation R.**

$$R1 \cup R2 = R$$

2. **The intersection of attributes of both the sub relations R1 and R2 must not be null, i.e., there should be some attributes that are present in both R1 and R2.**

$$R1 \cap R2 \neq \emptyset$$

3. **The intersection of attributes of both the sub relations R1 and R2 must be the superkey of R1 or R2, or both R1 and R2.**

$$R1 \cap R2 = \text{Super key of } R1 \text{ or } R2$$

Lossless Join Decomposition

<EmployeeProjectDetail>

Employee_Code	Employee_Name	Employee_Email	Project_Name	Project_ID
101	John	john@demo.com	Project103	P03
101	John	john@demo.com	Project101	P01
102	Ryan	ryan@example.com	Project102	P02
103	Stephanie	stephanie@abc.com	Project102	P02

Lossless Join Decomposition

Now, we decompose this relation into **EmployeeProject and ProjectDetail** relations as:

<EmployeeProject>

Employee_Code	Project_ID	Employee_Name	Employee_Email
101	P03	John	john@demo.com
101	P01	John	john@demo.com
102	P04	Ryan	ryan@example.com
103	P02	Stephanie	stephanie@abc.com

The primary key of the above relation is {Employee_Code, Project_ID}.

<ProjectDetail>

Project_ID	Project_Name
P03	Project103
P01	Project101
P04	Project104
P02	Project102

The primary key of the above relation is {Project_ID}.

Lossless Join Decomposition

Now, let's see if this is a lossless join decomposition by evaluating the rules discussed above:

Let's first check the EmployeeProject ∪ ProjectDetail:

<EmployeeProject ∪ ProjectDetail>

Employee_Code	Project_ID	Employee_Name	Employee_Email	Project_Name
101	P03	John	john@demo.com	Project103
101	P01	John	john@demo.com	Project101
102	P04	Ryan	ryan@example.com	Project104
103	P02	Stephanie	stephanie@abc.com	Project102

EmployeeProject ∪ ProjectDetail relation and it is the same as the original relation. So the **first condition holds.**

Lossless Join Decomposition

Now let's check the EmployeeProject ∩ ProjectDetail:

<EmployeeProject ∩ ProjectDetail>

Project_ID
P03
P01
P04
P02

As we can see this is not null, so the the second condition holds as well.

Also the **EmployeeProject ∩ ProjectDetail = Project_Id.** This is the **super key of the ProjectDetail** relation, so the third condition holds as well.

Now, since **all three conditions hold** for our decomposition, this is a **lossless join decomposition.**

Problem:

Consider a relation schema R (A , B , C , D) with the functional dependencies **A → B and C → D**. Determine whether the decomposition of R into **R1 (A , B) and R2 (C , D)** is lossless or lossy.

Condition-01:

According to condition-01, union of both the sub relations must contain all the attributes of relation R. So, we have-

$$R_1 (A , B) \cup R_2 (C , D)$$

$$= R (A , B , C , D)$$

Clearly, union of the sub relations contain all the attributes of relation R. Thus, condition-01 satisfies.

Problem:

Condition-02:

According to condition-02, intersection of both the sub relations must not be null.

So, we have-

$$R_1(A, B) \cap R_2(C, D)$$

$$= \Phi$$

Clearly, intersection of the sub relations is null.

So, condition-02 fails.

Thus, we conclude that the **decomposition is lossy**.

Dependency Preserving Decomposition

Dependency Preservation

A Decomposition D = { R1, R2, R3....Rn } of R is dependency preserving wrt a set F of Functional dependency if

$$(F1 \cup F2 \cup ... \cup Fm)+ = F+.$$

Consider a relation R is decomposed or divided into R1 with FD { f1 } and R2 with { f2 }, then there can be two cases:

f1 U f2 = F > Decomposition is dependency preserving.

f1 U f2 is a subset of F > Not Dependency preserving.

Normalization
or
Schema Refinement
or
Database design

Introduction to Normalization

Definition of Normalization

"Normalization is a process of designing a consistent database by minimizing redundancy and ensuring data integrity **through decomposition which is lossless**."

Schema Refinement

The Schema Refinement refers to refine the schema by using some technique. The best technique of schema refinement is **decomposition.**

Types of Anomalies : (Problems because of Redundancy)

There are three types of Anomalies produced in the database because of redundancy –

1. **Updation/Modification Anomaly**
2. **Insertion Anomaly**
3. **Deletion Anomaly**

Introduction to Normalization

1. **Normalization** is a process of organizing the data in the database.
2. It is a systematic approach of decomposing tables to eliminate data redundancy.
3. It was developed by **E. F. Codd.**
4. It is a step by step decomposition of complex records into simple records.
5. It is also called as Canonical Synthesis.

Introduction to Normalization

✓The Basic **Goal of Normalisation** is used to **eliminate redundancy**.

✓Redundancy refers to repetition of same data or duplicate copies of same data stored in different locations.

Normalization is used for mainly two purpose :

1. Eliminating redundant(useless) data.
2. Ensuring data dependencies make sense i.e. data is logically stored.

Features of Normalization

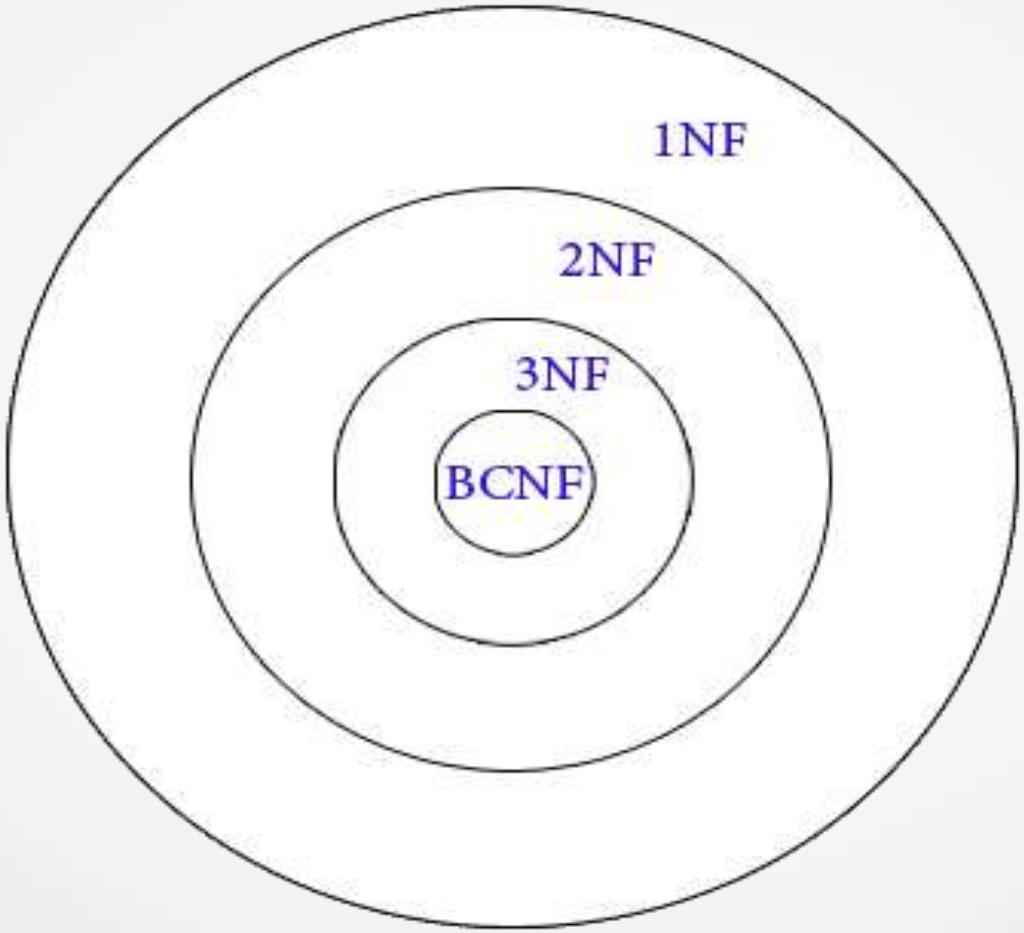
1. Normalization avoids the data redundancy.
2. It is a formal process of developing data structures.
3. It promotes the data integrity.
4. It ensures data dependencies make sense that means data is logically stored.
5. It eliminates the undesirable characteristics like Insertion, Updation and Deletion Anomalies.

Types of Normalization

Following are the types of Normalization:

1. *First Normal Form(1NF)*
2. *Second Normal Form(2NF)*
3. *Third Normal Form(3NF)*
4. *BCNF (Boyce – Codd Normal Form)*
5. *Fourth Normal Form(4NF)*
6. *Fifth Normal Form(5NF)*

Types of Normalization



First Normal Form(1NF)

First Normal Form(1NF)

1. First Normal Form (1NF) is a **simple form of Normalization**.

The requirements to satisfy the 1st NF:

1. **Each table has a primary key: minimal set of attributes which can uniquely identify a record**
2. **The values in each column of a table are atomic (No multi-value attributes allowed).**

Example::(1NF)

Example: Employee Table

ECode	Employee_Name	Department_Name
1	ABC	Sales, Production
2	PQR	Human Resource
3	XYZ	Quality Assurance, Marketing

Employee Table using 1NF

ECode	Employee_Name	Department_Name
1	ABC	Sales
1	ABC	Production
2	PQR	Human Resource
3	XYZ	Quality Assurance
3	XYZ	Marketing

Second Normal Form(2NF)

Second Normal Form(2NF)

Relation ***R*** is in Second Normal Form (2NF) only iff :

1. ***R* should be in 1NF and**
2. ***R* should not contain any *Partial Dependency***

Second Normal Form(2NF)

What is a Partial Dependency ?

Let ***R*** be a relational Schema and ***X,Y,A*** be the attribute sets over ***R***.

X: **Any Candidate Key**

Y: **Proper Subset of Candidate Key**

A: **Non Key Attribute If Y → A exists in R, then R is not in 2 NF.**

(Y → A) is a Partial dependency only if

Y: **Proper subset of Candidate Key**

A: **Non Prime Attribute.**

An attribute which is not part of candidate key is known as nonprime attribute.

Example : Employee Table using 1NF

ECode	Employee_Name	Employee_Age
1	ABC	38
1	ABC	38
2	PQR	38
3	XYZ	40
3	XYZ	40

Candidate Key: ECode, Employee_Name
Non prime attribute: Employee_Age

The above table is in 1NF. Each attribute has atomic values.

1. However, it is not in 2NF because non prime attribute Employee_Age is dependent on ECode alone, which is a proper subset of candidate key.

2. This violates the rule for 2NF

Example 1 :

Consider the relation ***Student(SID, Sname, Cname)*** which is in 1 NF (No MultiValuedAttributes) :

Student :

SID	Sname	Cname
S1	A	C
S1	A	C++
S2	B	C++
S2	B	DB
S3	A	DB

Example 1 :

{SID,Cname} : Primary Key

Functional Dependencies:

{SID,Cname} → Sname

SID → Sname

Partial Dependencies :

SID → Sname

{as SID is a Proper Subset of Candidate Key {SID,Cname}.

Example 1 :

Solution : Removal of Partial Dependency by creating separate table

⇓

R1 :	
SID	**Sname**
S1	A
S2	B
S3	A

SID : Primary Key

R2 :	
SID	**Cname**
S1	C
S1	C++
S2	C++
S2	DB
S3	DB

{SID,Cname} : Primary Key

Example 2 :

Consider the following table:

CAND_ID	SUBJECT_NO	SUBJECT_FEE
111	S1	1000
222	S2	1500
111	S4	2000
444	S3	1000
444	S1	1000
222	S5	2000

Example 2 :

In this table, several subjects have the same subject fee. There are three key observations here:

1. The SUBJECT_FEE cannot determine the values of CAND_NO or SUBJECT_NO independently;

2. The SUBJECT_FEE in combination with CAND_NO cannot determine the values of SUBJECT_NO;

3. The SUBJECT_FEE in combination with SUBJECT_NO cannot determine the values of CAND_NO;

Example 2 :

{SUBJECT_NO, CAND_ID} : **Primary Key**

Functional Dependencies:

{SUBJECT_NO, CAND_ID} → SUBJECT_FEE

SUBJECT_NO → SUBJECT_FEE

Partial Dependencies :

SUBJECT_NO → SUBJECT_FEE

{as SUBJECT_NO is a Proper Subset of Candidate Key {SUBJECT_NO, CAND_ID}

Example 2 :

Conclusion: The relation in this table is not in 2NF.

To convert this relation into 2NF, we split the table into two:

Table 1: CAND_NO, SUBJECT_NO
Table 2: SUBJECT_NO, SUBJECT_FEE.

Table 1

CAND_NO	SUBJECT_NO
111	S1
222	S2
111	S4
444	S3
444	S1
222	S5

Table 2

SUBJECT_NO	SUBJECT_FEE
S1	1000
S2	1500
S3	1000
S4	2000
S5	2000

Third Normal Form(3NF)

Third Normal Form(3NF)

Let *R* be the relational schema, *R* is in 3NF only if

1. ***R* should be in 2NF.**
2. ***R* should not contain *transitive dependencies*.**

What is a Transitive Dependency ?

Let ***R*** be a relational Schema and ***X,Y,Z*** be the attribute sets over ***R.***

If ***Y*** is functionally dependent on X ***(X → Y)***
and ***Z***is functionally dependent on Y ***(Y → Z)***
then ***X* is transitive dependent on *Z* *(X →Z)***

Example of 3NF :

Consider the relation **Sup_City(SID, Status, City)** :

Sup_City :

SID	Status	City
S1	30	Delhi
S2	10	Karnal
S3	40	Rohtak
S4	30	Delhi

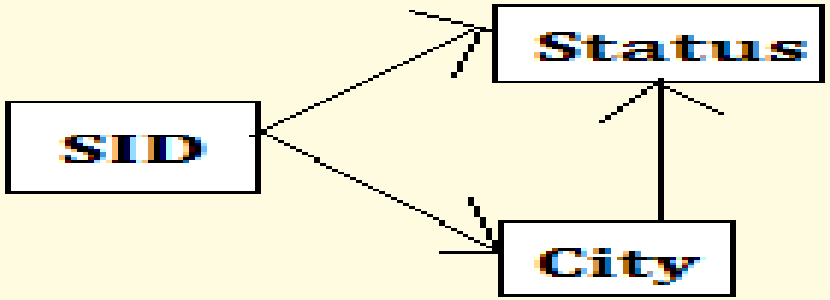
SID : Primary Key

Example of 3NF :

Sup_City :

SID	Status	City

FDD of Sup_City :



Transitive Dependency :

SID → City {As SID → City and City → Status}

Example of 3NF :

Solution :

Removal of Transitive Dependency by creating separate table

⇓

SC :	
SID	**City**
S1	Delhi
S2	Karnal
S3	Rohtak
S4	Delhi
SID : Primary Key	

CS :	
City	**Status**
Delhi	30
Karnal	10
Rohtak	40
City : Primary Key	

The relations SC and CS are in 3NF as they doesn't contain any transitive dependencies.

BCNF – Boyce Codd Normal Form

BCNF – Boyce Codd Normal Form

1. BCNF which stands for Boyce – Code Normal From is developed by **Raymond F. Boyce and E. F. Codd in 1974**.

2. BCNF is a **higher version of 3NF**.

3. It deals with the certain type of anomaly which is not handled by 3NF.

4. A table complies with **BCNF if it is in 3NF** and any attribute is fully functionally dependent that is A → B. (Attribute 'A' is determinant).

5. If **every determinant is a candidate key**, then it is said to be BCNF.

BCNF – Boyce Codd Normal Form

Let ***R*** be the relational schema, ***R*** is in BCNF only if :

1. ***R*** should be in 3NF.
2. Every Functional Dependency will have a Superkey on the LHS or all determinants are the superkeys.

Sometimes is BCNF is also referred as 3.5 Normal Form.

Example :

Consider the following relationship ***R(ABCD)*** having following functional dependencies

$$F = \{ A \rightarrow BCD,\ BC \rightarrow AD,\ D \rightarrow B \}$$

Above relationship is already in 3NF

Candidate Keys are :

$(A)^+ = \{ABCD\}$

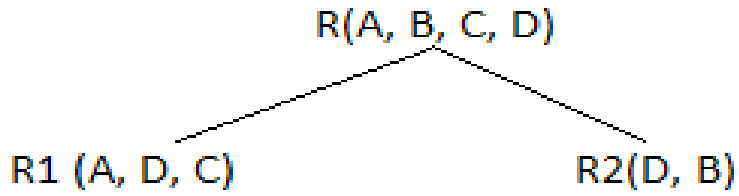
$(BC)^+ = \{BCAD\}$

Example :

Functional Dependency	Is FD in BCNF or not ?	Reason ?
A → BCD	Yes	A is a super key
BC → AD	Yes	BC is also a super key
D → B	No	D is not super key, it is part of key

Solution : Decomposition in BCNF

The relation ***R(ABCD)*** is decomposed into two relations ***R1*** and ***R2*** such that :



Breaking, table into two tables, one with A, D and C while the other with D and B.

Difference between 3NF and BCNF –

S.NO	3NF	BCNF
1.	It concentrates on Primary Key	It concentrates on Candidate Key.
2.	Redundancy is high as compared to BCNF	0% redundancy
3.	It may preserve all the dependencies	It may not preserve the dependencies.
4.	A dependency X → Y is allowed in 3NF if X is a super key or Y is a part of some key.	A dependency X → Y is allowed if X is a super key

4NF Normal Form

4NF Normal Form

A relation ***R*** is in Fourth Normal Form(4NF) if :

1. It is in ***BCNF***
2. It has ***no*** ***Multivalued Dependencies***

Multivalued Dependency

For a dependency A → B, **if for a single value of A, multiple value of B exists**, then the table may have multi-valued dependency.

A →→ B (B multi-valued depends on A)

Multivalued Dependency

A table is said to have multivalued dependency, if the following conditions are true,

1.A table should have atleast 3 columns for it to have a multivalued dependency.

2.And, for a relation R(A,B,C), if there is a multivalued dependency between, A and B, then B and C should be independent of each other.

Example

For example: Consider a bike manufacture company, which produces two colors (Black and white) in each model every year.

bike_model	manuf_year	color
M1001	2007	Black
M1001	2007	Red
M2012	2008	Black
M2012	2008	Red
M2222	2009	Black
M2222	2009	Red

Example

1.Here columns **manuf_year and color are independent** of each other and dependent on bike_model.

2.In this case **these two columns are said to be multivalued dependent on bike_model.**

These dependencies can be represented like this:

bike_model >> manuf_year

bike_model >> color

Fifth Normal Form / Projected Normal Form (5NF)

Fifth Normal Form / Projected Normal Form (5NF):

1. A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.

2. 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.

3. 5NF is also known as Project-join normal form (PJ/NF).

Fifth Normal Form / Projected Normal Form (5NF):

Example

SUBJECT	LECTURER	SEMESTER
Computer	Anshika	Semester 1
Computer	John	Semester 1
Math	John	Semester 1
Math	Akash	Semester 2
Chemistry	Praveen	Semester 1

Fifth Normal Form / Projected Normal Form (5NF):

•In the above table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2.

•In this case, combination of all these fields required to identify a valid data.

•Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL. But all three columns together acts as a primary key, so we can't leave other two columns blank.

•So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

Fifth Normal Form / Projected Normal Form (5NF):

P1

SEMESTER	SUBJECT
Semester 1	Computer
Semester 1	Math
Semester 1	Chemistry
Semester 2	Math

P2

SUBJECT	LECTURER
Computer	Anshika
Computer	John
Math	John
Math	Akash
Chemistry	Praveen

P3

SEMSTER	LECTURER
Semester 1	Anshika
Semester 1	John
Semester 1	John
Semester 2	Akash
Semester 1	Praveen

A surrogate key

or

synthetic key

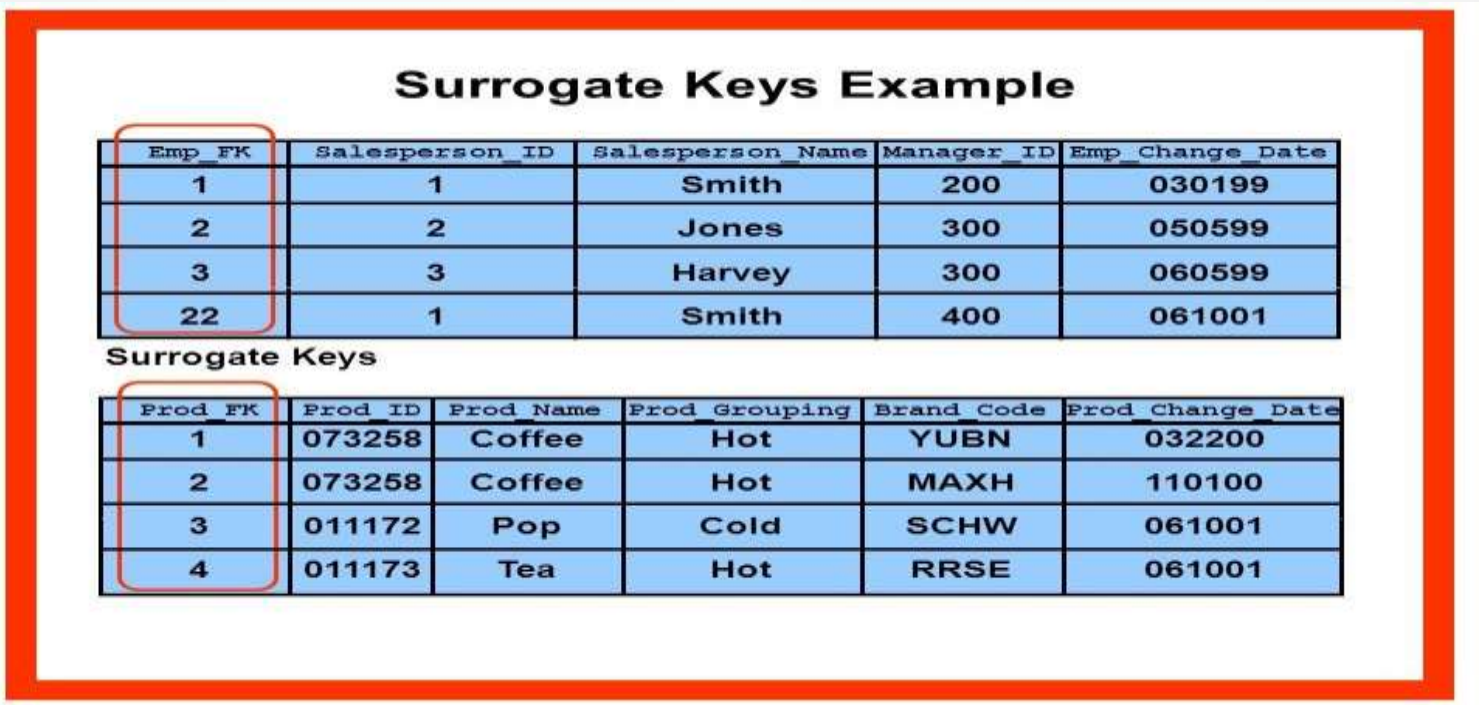
or

systemgenerated key

A surrogate key

A surrogate key is any column or set of columns that can be declared as the primary key instead of a "real" or natural key.

Sometimes there can be several natural keys that could be declared as the primary key, and these are all called candidate keys.



Surrogate Keys Example

Emp_FK	Salesperson_ID	Salesperson_Name	Manager_ID	Emp_Change_Date
1	1	Smith	200	030199
2	2	Jones	300	050599
3	3	Harvey	300	060599
22	1	Smith	400	061001

Surrogate Keys

Prod_FK	Prod_ID	Prod_Name	Prod_Grouping	Brand_Code	Prod_Change_Date
1	073258	Coffee	Hot	YUBN	032200
2	073258	Coffee	Hot	MAXH	110100
3	011172	Pop	Cold	SCHW	061001
4	011173	Tea	Hot	RRSE	061001

A surrogate key

A surrogate should have the following characteristics:

1. The value is unique systemwide, hence never reused
2. The value is system generated
3. The value is not manipulable by the user or application
4. The value contains no semantic meaning
5. The value is not visible to the user or application
6. The value is not composed of several values from different domains.